

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Мобильная разработка

Отчет

**Курсовая работа: Разработка прототипа Android
приложения Мессенджер**

Выполнил:

Фирсов Илья

Группа К3441

Проверил:

Шатинский Григорий Сергеевич

Санкт-Петербург

2026 г.

СОДЕРЖАНИЕ

1. Цели и задачи работы.....	3
Характеристики и используемые технологии:.....	3
2. Функциональные и нефункциональные требования.....	4
3. Архитектура и паттерны проектирования.....	5
4 Этапы разработки.....	5
4.1 Лабораторная работа 1: базовая структура приложения.....	5
4.1.1 Основная Activity и навигация.....	6
4.2 Лабораторная работа 2: MVVM и управление состоянием.....	7
4.2.1 Фрагмент «Профиль» и ViewModel пользователя.....	8
4.2.2 Слушатель изменений текста.....	10
4.2.3 Фрагмент «Настройки» и переключение темы.....	11
4.3 Лабораторная работа 3: работа с сетью и локальным хранилищем..	13
4.3.1 Список сообщений («Лента») и репозиторий данных.....	14
4.3.2 ViewModel и Repository.....	16
4.3.3 FeedFragment.....	17
4.3.4 Репозиторий и база данных.....	18
4.3.5 Фабрика FeedViewModelFactory.....	21
4.4 Лабораторная работа 4: UI, интерактивность и фоновая работа....	22
4.4.1 Фоновая синхронизация с WorkManager.....	23
4.4.2 Лайки и взаимодействие с сообщениями.....	25
4.4.3 Помощник уведомлений (NotificationHelper).....	26
5. Выводы.....	27

1. Цели и задачи работы

Цель проекта – разработка полнофункционального прототипа мессенджера на языке Kotlin с использованием современных архитектурных подходов. Приложение должно обеспечивать обмен сообщениями через публичный API с возможностью локального хранения, а также содержать экраны «Лента», «Профиль» и «Настройки» в единой Activity с нижней навигацией. Проект реализован в четыре этапа (лабораторные работы), каждый из которых добавлял новые возможности и компоненты.

Характеристики и используемые технологии:

- Язык и платформа: Android (минимальная версия API 24), язык Kotlin.
- Архитектура: MVVM (Model-View-ViewModel) с разделением на слои View, ViewModel и Model (Repository/DAO).
- Jetpack-компоненты:
 1. ViewModel и LiveData для управления состоянием UI и реактивного обновления.
 2. Navigation Component для организации навигации между фрагментами внутри единой Activity (использован BottomNavigationView и NavController).
 3. Room для локальной базы данных сообщений (Entity, Dao, Database).
 4. WorkManager для фоновой синхронизации данных.
 5. Material Design: CardView, FloatingActionButton, AppBarLayout, BottomNavigationView и другие компоненты Material UI.
- Сетевые библиотеки: Retrofit + Gson Converter для получения данных (сообщений) из публичного API.
- Репозиторий и паттерны: Repository как единая точка доступа к данным, шаблон Factory для создания ViewModel с зависимостями.
- Другие: Корутинная поддержка (viewModelScope) для асинхронных операций, компоненты для работы с разрешениями и уведомлениями (NotificationCompat).

2. Функциональные и нефункциональные требования

- Основные экраны:
 1. Лента: отображает список сообщений, полученных из удалённого сервиса и сохранённых в локальной базе. Есть кнопка «Обновить» для ручной синхронизации.
 2. Профиль: показывает имя и статус пользователя (по умолчанию заполнены «Firsov Iliа» и «Studying kotlin»). Пользователь может редактировать эти поля; данные сохраняются во ViewModel и не теряются при повороте экрана.
 3. Настройки: содержит переключатель темы (светлая/тёмная); состояние переключателя хранится в ViewModel и применяется ко всему приложению.
- Навигация: нижняя навигационная панель (BottomNavigationView) переключает между тремя фрагментами. Навигация настроена через NavController и граф навигации (Navigation Component).
- Синхронизация: при старте приложения и по нажатию кнопки «Обновить» загружаются сообщения через Retrofit из публичного API <https://dummyjson.com/posts>, и результат сохраняется в Room-базу. При отсутствии сети данные берутся из локальной базы.
- Фоновая работа: с помощью WorkManager настроен периодический запуск SyncWorker, который каждые N часов обновляет список сообщений и при удачной синхронизации вызывает уведомление «Новые данные получены» (Notification).
- Интерактивность: кастомный RecyclerView.Adapter для сообщений отображает аватар, имя (title) и текст (body) сообщения, реализовано «лайканье»: при клике на иконку сердце меняется на заполненное (и наоборот).
- Логирование: в консоль (Log.d) выводятся сообщения о жизненном цикле фрагментов и ViewModel (создание/уничтожение) для отладки и контроля.
- Нефункциональные требования: код проекта организован по принципам чистой архитектуры (MVVM, однообразные интерфейсы, Repository), приложение должно быть отзывчивым, сохранять

состояние при повороте экрана, корректно обрабатывать доступ к сети, разрешения (уведомления) и переход в оффлайн/онлайн.

3. Архитектура и паттерны проектирования

Приложение спроектировано по паттерну MVVM. Каждое View (Activity/Fragment) управляет только UI и наблюдает за данными во ViewModel, не содержит бизнес-логики. ViewModel хранит состояние экрана и обращается к Repository для получения данных. Репозиторий, в свою очередь, инкапсулирует источники данных: сетевой API через Retrofit и локальную базу данных через Room (DAO).

ViewModel получает LiveData из Repository, которая комбинирует данные из Room и сети. Для сетевых запросов используется Retrofit Client, результаты которого сохраняются в Room через DAO. При изменении данных ViewModel обновляет LiveData, и UI-фрагмент автоматически реагирует на эти изменения. Таким образом обеспечивается разбиение ответственности и реактивность интерфейса. Дополнительно применены шаблоны Factory (для создания ViewModel с параметрами) и Singleton (для RetrofitClient и базы данных).

4 Этапы разработки

4.1 Лабораторная работа 1: базовая структура приложения

- Создан один главный MainActivity с BottomNavigationView и NavHostFragment (Navigation Graph) на три вкладки: «Лента», «Профиль», «Настройки».
- Реализованы три фрагмента-заглушки с простым текстом и выводом сообщений Log в методах жизненного цикла (onCreate, onStart, onResume и др.) для контроля.
- Настроена навигация через NavController (компоненты Android Navigation). Добавлены необходимые разрешения в AndroidManifest (INTERNET, POST_NOTIFICATIONS, ACCESS_NETWORK_STATE). В проекте активирован ViewBinding для работы с элементами UI.
- Результат: приложение запускается без ошибок, интерфейс понятен (настройки темы, профиль, лента), навигация между разделами

работает по стандартному протоколу. Код организован модульно, подготовлен для расширения.

4.1.1 Основная Activity и навигация

В MainActivity используется ViewBinding для инициализации интерфейса, а также настраивается навигация через BottomNavigationView. В коде это реализовано так: сначала раздуваем разметку и устанавливаем её в качестве содержимого экрана, затем связываем NavController с нижней навигацией. Код фрагмента onCreate():

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
  
    // Инициализируем ViewBinding и устанавливаем  
    // содержимое Activity.  
    binding =  
        ActivityMainBinding.inflate(layoutInflater)  
    setContentView(binding.root)  
  
    // Настраиваем навигацию: связываем  
    // BottomNavigationView с NavController.  
    val navHostFragment = supportFragmentManager  
        .findFragmentById(R.id.nav_host_fragment) as  
        NavHostFragment  
    val navController = navHostFragment.navController  
  
    binding.bottomNavigation.setupWithNavController(navController)  
  
    if (Build.VERSION.SDK_INT >=  
        Build.VERSION_CODES.TIRAMISU) {
```

```

        if
        (checkSelfPermission (Manifest.permission.POST_NOTIFICATIONS) != PackageManager.PERMISSION_GRANTED) {

            requestPermissions (arrayOf (Manifest.permission.POST_NOTIFICATIONS), REQ_CODE_NOTIF)

        }

    }

    Log.d (TAG, "onCreate: MainActivity создана")
}

```

- ViewBinding (ActivityMainBinding.inflate(...)) упрощает доступ к элементам разметки без findViewById.
- NavController ищется по идентификатору навигационного хоста (R.id.nav_host_fragment), после чего BottomNavigationView связывается с контроллером через setupWithNavController(...). Это обеспечивает автоматическую навигацию между фрагментами при клике по иконкам.
- Для Android 13+ запрашивается разрешение на показ уведомлений (POST_NOTIFICATIONS) при необходимости.

Таким образом, в MainActivity реализован стандартный каркас: установка основного макета через ViewBinding и организация навигации по вкладкам.

4.2 Лабораторная работа 2: MVVM и управление состоянием

- Введена архитектура MVVM: для экранов «Профиль» и «Настройки» созданы соответствующие ViewModel (ProfileViewModel, SettingsViewModel). Данные профиля (имя и статус) и тема хранятся во ViewModel и обновляются через LiveData.
- Реализован экран Профиль: поля EditText для имени и статуса. В профиле используется doOnTextChanged для обновления ViewModel

при вводе, а фрагмент подписывается на LiveData из ViewModel и устанавливает текст в поля, избегая циклов обновления.

- Реализован экран Настройки: переключатель темы (Switch). Фрагмент подписывается на isDarkTheme: LiveData<Boolean> из SettingsViewModel и обновляет AppCompatActivity в зависимости от значения (длинное включение тёмной темы). При клике на переключатель вызывается метод viewModel.setDarkTheme(isChecked).
- Благодаря MVVM при повороте экрана данные не теряются (ViewModel сохраняет их). Во ViewModel профиля и настроек добавлены сообщения Log.d для отслеживания жизненного цикла.

4.2.1 Фрагмент «Профиль» и ViewModel пользователя

Фрагмент ProfileFragment показывает данные профиля (имя и статус) и позволяет их изменять. Для хранения состояния используется класс ProfileViewModel с двумя полями MutableLiveData<String> (_name и _status), которые экспортируются наружу как LiveData только для чтения. Пример кода из ProfileViewModel:

```
class ProfileViewModel : ViewModel() {  
    private val tag = "ProfileViewModel"  
  
    // Внутренние изменяемые поля  
  
    private val _name =  
MutableLiveData<String>("Firsov Ilia")  
  
    private val _status =  
MutableLiveData<String>("Studying kotlin")  
  
    // Публичные LiveData только на чтение  
  
    val name: LiveData<String> = _name  
    val status: LiveData<String> = _status  
  
    init {  
        Log.d(tag, "init: ViewModel создана")  
    }  
}
```



```

    }

    fun updateName(newName: String) {

        Log.d(tag, "updateName: $newName")

        _name.value = newName

    }

    fun updateStatus(newStatus: String) {

        Log.d(tag, "updateStatus: $newStatus")

        _status.value = newStatus

    }

    override fun onCleared() {

        Log.d(tag, "onCleared: ViewModel будет уничтожена")

        super.onCleared()

    }

}

```

Ключевые моменты: публичные поля `name` и `status` — это неизменяемые `LiveData`, на которые может подписаться UI. Метод `updateName()` устанавливает новое значение в `_name`, что автоматически оповещает подписчиков. Использование `LiveData` гарантирует, что изменения будут учтены с привязкой к жизненному циклу фрагмента.

В `ProfileFragment` выполняется две операции:

- Подписка на `LiveData`: фрагмент наблюдает за свойствами `name` и `status` `ViewModel` с помощью `observe(viewLifecycleOwner)`. При изменении данных обновляются соответствующие поля ввода.

Например:

```
viewModel.name.observe(viewLifecycleOwner) {
    name ->
        Log.d(tag, "observe name: $name")
        // Чтобы не зациклиться, ставим текст только
        // если он реально отличается
        if (binding.etProfileName.text.toString() !=
            name) {
            binding.etProfileName.setText(name)
        }
    }
}
```

Здесь observe обеспечивает автоматическую передачу последних данных во View. В соответствии с реактивной моделью, изменения, сделанные в ViewModel, сразу отражаются на UI. То есть когда мы вызываем updateName(...), LiveData внутри ViewModel меняется, и подписанный фрагмент получает новое значение и перерисовывает поле ввода.

4.2.2 Слушатель изменений текста

С помощью расширения doOnTextChanged (androidx.core.widget) фрагмент отслеживает ввод пользователя в EditText и передаёт новые значения во ViewModel:

```
binding.etProfileName.doOnTextChanged { text, _,
_, _ ->
    viewModel.updateName(text?.toString().orEmpty())
}

binding.etProfileStatus.doOnTextChanged { text,
_, _, _ ->
    viewModel.updateStatus(text?.toString().orEmpty())
}
```

```
}
```

Каждый раз при изменении текста вызывается метод `updateName` (или `updateStatus`) `ViewModel`, который обновляет соответствующее `LiveData`. Это, в свою очередь, приводит к срабатыванию блока `observe` и обновлению UI. Таким образом реализуется двунаправленная связь: `LiveData` обновляется из `View`, и `View` автоматически обновляется при изменении `LiveData`.

4.2.3 Фрагмент «Настройки» и переключение темы

Фрагмент `SettingsFragment` отвечает за переключение тёмной/светлой темы. Здесь также используется `ViewModel` (`SettingsViewModel`) и `LiveData`. Код `ViewModel` схож с предыдущим примером:

```
class SettingsViewModel : ViewModel() {

    private val tag = "SettingsViewModel"

    // true = тёмная тема, false = светлая

    private val _isDarkTheme =
MutableLiveData<Boolean>(false)

    val isDarkTheme: LiveData<Boolean> = _isDarkTheme

    init {

        Log.d(tag, "init: ViewModel настроек
создана")

    }

    fun setDarkTheme(enabled: Boolean) {

        Log.d(tag, "setDarkTheme: $enabled")

        _isDarkTheme.value = enabled
    }
}
```

```

    }

    override fun onCleared() {
        Log.d(tag, "onCleared: ViewModel настроек
будет уничтожена")

        super.onCleared()
    }
}

```

Во фрагменте SettingsFragment выполняются аналогичные шаги:

- Подписка на LiveData: наблюдаем за isDarkTheme. При изменении этого флага обновляем переключатель (Switch) и применяем новую тему ко всему приложению:

```

// 1. Подписываемся на LiveData темы
viewModel.isDarkTheme.observe(viewLifecycleOwner
) { isDark ->
    Log.d(tag, "observe isDarkTheme: $isDark")

    // Обновляем переключатель только если нужно,
    чтобы не дергать лишний раз listener
    if (binding.switchDarkMode.isChecked !=
isDark) {
        binding.switchDarkMode.isChecked = isDark
    }

    // Применяем тему во всём приложении
    AppCompatActivity.setDefaultNightMode(
        if (isDark)
AppCompatActivity.MODE_NIGHT_YES
        else AppCompatActivity.MODE_NIGHT_NO
    )
}

```

```
    )  
}
```

Здесь используется тот же механизм LiveData: фрагмент подписывается с учётом жизненного цикла. При изменении `_isDarkTheme` в модели фрагмент автоматически получает новое значение и мгновенно переключает тему приложения.

Слушатель переключателя: при изменении состояния Switch сохраняем выбранную тему во ViewModel:

```
binding.switchDarkMode.setOnCheckedChangeListener  
r { _, isChecked ->  
  
    Log.d(tag, "switchDarkMode changed by user:  
$isChecked")  
  
    viewModel.setDarkTheme(isChecked)  
  
}
```

Это вызывает метод `setDarkTheme(enabled)` во `SettingsViewModel`, который записывает новое значение в `MutableLiveData`. В результате блок `observe` выше сработает и обновит UI. Использование LiveData здесь обеспечивает безопасное обновление UI без утечек и с учётом жизненного цикла фрагмента.

Таким образом, фрагменты «Профиль» и «Настройки» демонстрируют основной паттерн MVVM: ViewModel содержит состояние в LiveData, а View (фрагмент) подписывается на эти данные и обновляет интерфейс автоматически.

4.3 Лабораторная работа 3: работа с сетью и локальным хранилищем

- Добавлен `MessageRepository` – класс-репозиторий, который получает данные из публичного API (`MessageApi` через `Retrofit`) и сохраняет их в локальную базу `Room`. Репозиторий инжектируется в ViewModel экрана «Лента».

- Retrofit-клиент: создан объект `RetrofitClient.api` с базовым URL `https://dummyjson.com/`. Интерфейс `MessageApi` имеет метод `@GET("posts") suspend fun getPosts(): PostsResponseDto`.
- Room: сущность `MessageEntity(id, title, body)` и DAO `MessageDao` с методом `observeAll(): LiveData<List<MessageEntity>>` (запрос получает все сообщения, отсортированные по убыванию `id`).
- При запуске приложения (и нажатии «Обновить») `FeedViewModel` в `viewModelScope` вызывает `repository.refresh()`, который выполняет сетевой запрос, маппит результат в список сущностей и сохраняет через `dao.upsertAll(...)`. После сохранения `LivData` из DAO автоматически обновляет список на экране.
- Отображение списка: во фрагменте `FeedFragment` используется `RecyclerView` с `LinearLayoutManager` и кастомный `MessagesAdapter`. Данные наблюдаются через `viewModel.messages.observe(...)` – список сообщений подставляется в адаптер. При отсутствии сети приложение переключается на отображение локального кеша (логика проверки сети дана через `ConnectivityManager`).

4.3.1 Список сообщений («Лента») и репозиторий данных

Следующий экран приложения – лента новостей (`FeedFragment`) с `RecyclerView`, показывающим список сообщений. Здесь задействован более сложный MVVM-поток с `Repository`, `Room` и `Retrofit`.

Для отображения списка сообщений используется класс `MessagesAdapter`, унаследованный от `ListAdapter` (использует `DiffUtil` для эффективного обновления списка). Пример фрагмента кода адаптера:

```
class MessagesAdapter : ListAdapter<MessageEntity,
MessagesAdapter.VH>(Diff) {

    override fun onCreateViewHolder(parent:
ViewGroup, viewType: Int): VH {

        val binding =
MessageBinding.inflate(LayoutInflater.from(parent.co
ntext), parent, false)

        return VH(binding)
```

```
    }

    override fun onBindViewHolder(holder: VH,
position: Int) {

        holder.bind(getItem(position))

    }

    inner class VH(private val binding:
MessageBinding) :
RecyclerView.ViewHolder(binding.root) {

        fun bind(item: MessageEntity) {

binding.ivAvatar.setImageResource(R.mipmap.ic_launch
er_round)

            binding.tvTitle.text = item.title

            binding.tvBody.text = item.body

        }

    }

    private object Diff :
DiffUtil.ItemCallback<MessageEntity>() {

        override fun areItemsTheSame(oldItem:
MessageEntity, newItem: MessageEntity) = oldItem.id
== newItem.id

        override fun areContentsTheSame(oldItem:
MessageEntity, newItem: MessageEntity) = oldItem ==
newItem

    }

}
```

Адаптер принимает объекты MessageEntity (сущность из Room) и связывает их с вёрсткой элемента списка.

4.3.2 ViewModel и Repository

FeedViewModel содержит логику экрана новостей и работает через репозиторий MessageRepository. Его код:

```
class FeedViewModel(  
    private val repository: MessageRepository  
) : ViewModel() {  
    private val tag = "FeedVM"  
  
    val messages: LiveData<List<MessageEntity>> =  
        repository.observeMessages()  
  
    fun refresh() {  
        Log.d(tag, "refresh: clicked")  
  
        viewModelScope.launch {  
            try {  
                repository.refresh()  
                Log.d(tag, "refresh: success")  
            } catch (e: Exception) {  
                Log.d(tag, "refresh: failed ->  
${e.message}")  
            }  
        }  
    }  
}
```


- Поле `messages` представляет собой `LiveData<List<MessageEntity>>`, получаемое из репозитория. Метод `repository.observeMessages()` возвращает `LiveData` из Room (см. DAO ниже), благодаря чему любые изменения в БД автоматически транслируются во `ViewModel` и далее в UI.
- Метод `refresh()` запускается в корутине (через `viewModelScope`) и вызывает `repository.refresh()`, который загружает новые сообщения из сети и сохраняет их в БД. В результате после сохранения БД через `LiveData` уведомит фрагмент, и список обновится автоматически.

4.3.3 FeedFragment

Во `FeedFragment` настраивается `RecyclerView` и подписка на `messages`. Ключевой код во `onViewCreated`:

```
binding.rvMessages.layoutManager =
    LinearLayoutManager(requireContext())

binding.rvMessages.adapter = adapter

viewModel.messages.observe(viewLifecycleOwner) {
    items ->

        Log.d(tag, "messages observed: ${items.size}")

        adapter.submitList(items)
    }

val online = isOnline(requireContext())

binding.tvOffline.visibility = if (online) View.GONE
    else View.VISIBLE

binding.btnRefresh.isEnabled = online

binding.btnRefresh.setOnClickListener {

    val nowOnline = isOnline(requireContext())
```

```

        binding.tvOffline.visibility = if (nowOnline)
View.GONE else View.VISIBLE

        binding.btnRefresh.isEnabled = nowOnline

        if (nowOnline) {

            viewModel.refresh()

        }

    }

    viewModel.refresh()

```

1. Настраивается RecyclerView с MessagesAdapter.
2. Подписка на LiveData: при получении нового списка items вызывается adapter.submitList(items). Благодаря этому RecyclerView автоматически обновляет отображение. (Эту реактивную связку обеспечивает LiveData из ViewModel.)
3. Если нажать «Обновить», проверяем наличие сети (метод isOnline). При доступности интернета вызываем viewModel.refresh(), что триггерит загрузку из API.
4. При старте фрагмента сразу вызывается viewModel.refresh(), чтобы заполнить базу данными при инициализации.

4.3.4 Репозиторий и база данных

MessageRepository связывает сетевой API (Retrofit) и локальную БД (Room):

```

class MessageRepository(

    private val api: MessageApi,

    private val dao: MessageDao

```

```

) {

    private val tag = "MessageRepository"

    fun observeMessages() = dao.observeAll()

    suspend fun refresh() {

        Log.d(tag, "refresh: start")

        val response = api.getPosts()

        val entities = response.posts.map {

            MessageEntity(

                id = it.id,

                title = it.title,

                body = it.body

            )

        }

        dao.upsertAll(entities)

        Log.d(tag, "refresh: saved ${entities.size}")

    }

}

```

- observeMessages() возвращает LiveData<List<MessageEntity>> из DAO – это даёт реактивный поток данных из локальной базы. При изменении таблицы «messages» Room автоматически оповещает наблюдателей.
- В refresh() выполняется сетевой запрос api.getPosts() (через RetrofitClient), полученные DTO-преобразуются в сущности MessageEntity, после чего вызывается dao.upsertAll(entities) для сохранения в БД. Конфликтные вставки обрабатываются стратегией

REPLACE (в DAO, см. ниже).

Сущность и DAO Room:

```
@Entity(tableName = "messages")  
  
data class MessageEntity(  
    @PrimaryKey val id: Int,  
    val title: String,  
    val body: String  
)
```

```
@Dao  
  
interface MessageDao {  
  
    @Query("SELECT * FROM messages ORDER BY id DESC")  
    fun observeAll(): LiveData<List<MessageEntity>>  
  
    @Insert(onConflict = OnConflictStrategy.REPLACE)  
    suspend fun upsertAll(items: List<MessageEntity>)  
}
```

- MessageEntity описывает таблицу messages с полями id, title, body.
- В DAO метод observeAll() возвращает LiveData со списком всех сообщений, упорядоченных по убыванию id. Благодаря LiveData мы получаем реактивное обновление UI при каждом изменении таблицы.

- Метод upsertAll с стратегией REPLACE сохраняет список сообщений в БД, заменяя существующие по ключу id.

Инициализация Room и Retrofit происходит в классе App (наследнике Application):

```
val db = Room.databaseBuilder(  
    applicationContext,  
    AppDatabase::class.java,  
    "lab1_db"  
) .build()  
  
messageRepository = MessageRepository(  
    api = RetrofitClient.api,  
    dao = db.messageDao()  
)
```

И настроена периодическая синхронизация через WorkManager. Таким образом, архитектура «репозиторий + Room + LiveData» обеспечивает устойчивое состояние и реактивный поток данных: при каждом обновлении таблицы список автоматически приходит во ViewModel, а View перерисовывает список без дополнительного кода синхронизации.

4.3.5 Фабрика FeedViewModelFactory

Чтобы передавать зависимость MessageRepository в FeedViewModel, используется специальная фабрика, реализующая интерфейс ViewModelProvider.Factory. Без этого Android не позволяет создать ViewModel с параметрами в конструкторе.

```
class FeedViewModelFactory(  

```

```

        private val repository: MessageRepository
    ) : ViewModelProvider.Factory {

        override fun <T : ViewModel>
create(modelClass: Class<T>): T {

            if
(modelClass.isAssignableFrom(FeedViewModel::class.java)) {

                @Suppress("UNCHECKED_CAST")

                return FeedViewModel(repository) as T

            }

            throw IllegalArgumentException("Unknown
ViewModel class: ${modelClass.name}")

        }

    }

```

Использование фабрики позволяет корректно инициализировать FeedViewModel в фрагменте.

4.4 Лабораторная работа 4: UI, интерактивность и фоновая работа

- Интерфейс: улучшен внешний вид списка сообщений. Каждая ячейка – карточка (CardView) с аватаром (иконка приложения), именем (title) и текстом (body). Добавлен FloatingActionButton для некой будущей функции (например, создания сообщения). Используются компоненты AppBarLayout и Material-стили.
- Adapter с «лайками»: в MessagesAdapter реализовано хранение множества likedIds (set of Int). В onBindViewHolder определяется, лайкнуто ли сообщение (по id). Иконка ivLike меняется на заполненную или пустую сердечко по нажатию, и идентификатор

добавляется/удаляется из множества. Таким образом пользователь может отмечать сообщения «избранными».

- **WorkManager**: настроен **SyncWorker : CoroutineWorker**, который в **doWork()** выполняет **app.messageRepository.refresh()** и возвращает **Result.success()** или **Result.retry()**. В **MainActivity** (или при инициализации приложения) создан периодический запрос через **WorkManager.enqueuePeriodicWork**, запускающий синхронизацию каждый фиксированный интервал.
- **Уведомления**: при успешной фоновой синхронизации вызывается метод **NotificationHelper.show(context, "Новые данные получены")**. Здесь проверяется канал уведомлений (создаётся при необходимости) и разрешения (особенно для Android 13+ проверяется **POST_NOTIFICATIONS**). В **NotificationHelper** строится **NotificationCompat.Builder** с заголовком и текстом, а затем уведомление выводится через **NotificationManagerCompat**.
- **Разрешения и системные события**: при запуске приложения проверяется и запрашивается разрешение на показ уведомлений. Приложение реагирует на переход в оффлайн (проверка наличия сети) и блокирует кнопку «Обновить», показывая при этом сообщение «Оффлайн» (видимость **TextView**).

4.4.1 Фоновая синхронизация с WorkManager

Для автоматического обновления списка сообщений используется **WorkManager** и класс **SyncWorker**. В **App** создаётся периодический **WorkRequest**:

```
val constraints = Constraints.Builder()

    .setRequiredNetworkType(NetworkType.CONNECTED)

    .build()

val request =
    PeriodicWorkRequestBuilder<SyncWorker>(10,
    TimeUnit.MINUTES)

    .setConstraints(constraints)

    .build()
```

```

WorkManager.getInstance(this).enqueueUniquePeriodicWork(
    "messages_sync",
    ExistingPeriodicWorkPolicy.KEEP,
    request
)

```

- Периодическая задача SyncWorker ставится с интервалом 10 минут и условием наличия интернет-соединения (setRequiredNetworkType(NetworkType.CONNECTED)), чтобы не расходовать трафик при оффлайне.
- Метод enqueueUniquePeriodicWork с политикой KEEPER гарантирует, что будет запущен только один такой уникальный воркер («messages_sync»). Даже если приложение попытается поставить ещё одну такую задачу, предыдущая сохранится (аналогично примеру из документации).

Класс SyncWorker просто вызывает app.messageRepository.refresh(), то есть повторно загружает и сохраняет данные в БД:

```

class SyncWorker(
    appContext: Context,
    params: WorkerParameters
) : CoroutineWorker(appContext, params) {
    override suspend fun doWork(): Result {
        return try {
            val app = applicationContext as App
            app.messageRepository.refresh()
        }
    }
}

```



```

        Result.success()

    } catch (e: Exception) {

        Result.retry()

    }

}

}

```

Таким образом, `WorkManager` обеспечивает регулярный фоновый сбор данных без участия пользователя. Периодическая работа задаётся по аналогии с примером разработчиков Android, гарантируя единичное запланированное выполнение.

4.4.2 Лайки и взаимодействие с сообщениями

Для повышения интерактивности интерфейса была реализована возможность «лайкать» сообщения. Эта функция работает локально — без записи в базу или сеть — и реализована на уровне адаптера сообщений. Логика заключается в хранении списка ID понравившихся сообщений в множестве `likedIds` и обновлении иконки лайка при каждом клике:

```

// Определяем, лайкнуто ли сообщение

val liked = likedIds.contains(item.id)

binding.ivLike.setImageResource(

    if (liked) R.drawable.ic_favorite_filled else
R.drawable.ic_favorite_border

)

binding.ivLike.setOnClickListener {

    if (likedIds.contains(item.id)) {

        likedIds.remove(item.id)
    }
}

```

```

        } else {

            likedIds.add(item.id)

        }

        binding.ivLike.setImageResource(

            if (likedIds.contains(item.id))
R.drawable.ic_favorite_filled else
R.drawable.ic_favorite_border

        )

    }

```

Этот механизм позволяет визуально отмечать понравившиеся сообщения. Он иллюстрирует работу с состоянием отдельных элементов списка, а также обработку пользовательского ввода в RecyclerView.

4.4.3 Помощник уведомлений (NotificationHelper)

Для отправки уведомлений при успешной фоновой синхронизации создан отдельный объект NotificationHelper. Он автоматически создаёт NotificationChannel (начиная с Android 8.0) и выводит уведомление:

```

object NotificationHelper {

    private const val CHANNEL_ID = "sync_channel"

    private const val NOTIFICATION_ID = 1

    fun show(context: Context, text: String) {

        createChannelIfNeeded(context)

        if (Build.VERSION.SDK_INT >=
Build.VERSION_CODES.TIRAMISU) {

            val granted =
context.checkSelfPermission(Manifest.permission.POST
_NOTIFICATIONS) ==

```

```

PackageManager.PERMISSION_GRANTED

        if (!granted) return
    }

    val notification =
NotificationCompat.Builder(context, CHANNEL_ID)

        .setSmallIcon(R.mipmap.ic_launcher)

.setContentTitle(context.getString(R.string.app_name
))

        .setContentText(text)

        .setAutoCancel(true)

.setPriority(NotificationCompat.PRIORITY_HIGH)

        .build()

NotificationManagerCompat.from(context).notify(NOTIFICATION_ID, notification)

    }

```

5. Выводы

В ходе работы были выполнены следующие задачи:

- Настройка навигации и основного экрана: используем BottomNavigationView с NavController для переключения между фрагментами (Новости, Профиль, Настройки).

- Использование ViewModel и LiveData: храним состояние (имя, статус, тема) во ViewModel-фрагментах и наблюдаем за ними во View. Это обеспечивает автоматическое восстановление данных при смене конфигурации (например, поворот экрана) и безопасную передачу данных в UI.
- Реактивный UI: при вводе новых данных (редактировании профиля или переключении темы) изменения сразу передаются в ViewModel и через LiveData возвращаются во фрагмент, обновляя интерфейс без ручной логики синхронизации.
- Работа с данными через Repository и Room: реализован репозиторий MessageRepository, который получает данные от REST API (Retrofit) и сохраняет их в локальную базу Room. Библиотека Room упрощает SQL (безопасность типов, LiveData-обновление). Список сообщений сохраняется между сессиями, а LiveData автоматически обновляет View при каждом изменении таблицы.
- Фоновая синхронизация: настроен WorkManager для периодического обновления данных (через SyncWorker), что поддерживает приложение в актуальном состоянии даже при длительном отсутствии активности со стороны пользователя.

В результате архитектура приложения разделена по слоям (MVVM + Repository), что делает код понятным, модульным и легко расширяемым. Все компоненты работают в едином реактивном потоке: View только отображает данные и реагирует на события, ViewModel хранит и обрабатывает состояние, а Repository управляет источниками данных. Такой подход обеспечивает устойчивость приложения и удобство тестирования, а также соответствует современным рекомендациям по разработке Android-приложений.